BANT Design Document

Our project takes an OOP approach to development, breaking out our application in 14 current files and a number of other technological dependencies.

Application.java:

Acts as the primary launch point for the project, building the application by launching the other files, starting from saveSelect.fxml and the other SceneBuilder files. The application.java could be thrown into another file but that would only act to further complicate the file structure and what file actually does what. In terms of our application this file acts as the main method.

CurrentPlayerSessionHelperClass.java:

While simple this helper class works to create a current player, the player.java splits the database configuration and this helper class allows us to access the player during their run, if the end user decides to switch to another save this method helps to create a seamless transition for the end user. This calls the player.java and makes an instance of that called currentPlayer, then a simple setter and getter act as the portal to the current session. Now that we are further along in the project, I feel that this could have been added to the Player.java and then called from there but it works well as a separate file, and we have never had any problems with it. When I made it, I was worried about violating naming conventions but in hindsight that would have mattered little.

DatabaseConnection.java:

This chunky file is what connects to our SQLite database, it finds the table with the findsTable method, it finds the primary key with the findsPrimaryKey method. The loadPlayer method takes in the savelot that the end user is currently in, then targets both the table and primary key along with selecting everything from the table that targets it. Once the SQL query has been executed the if statement triggers and returns all of the contents of the database. This is all wrapped up in a try catch to make sure that any errors that occur we know where they occurred. The updateMethodForItems could have probably also gone in MapController.java but it works here too. This method works similarly to the loadPlayer method, in fact if we had more time we could probably finds a way to put them together but as it is now they work well enough. This method is what is responsible for a majority of the methods in MapController.java which themselves tie to many of our backend buttons. Finally, the grabbingInfo method acts as a helper method for the

Enemy.java:

This is the enemy class responsible for the logic behind what the player fights, health, damage and the dialog options behind the interaction. The enemy class also contains a enemyEvent, which handles interaction with the player and the enemy. The player is thrown into a loop in a turn-based system, contained in a loop, in which they must attack or flee. If player attacks, enemy may be hit or player misses. This is controlled by a random value within a given range. Certain values determine a hit and others a miss. The enemy also can hit or miss and is also determined by a random value in a range.
If a player is hit, their health is affected.

eventDriver.java:

This is a testing main function for the NPC class

GameplayController.java/Launcher.java:

The GameplayController method calls the gameplay.fxml file and will hold the backend logic for the players' gameplay options. The Launcher.java file does not do anything at the moment and might be removed.

MapController.java:

This is the backend logic that was being referenced in the DatabaseConnection.java paragraph. All the methods for adding and removing the different items along with the functionality for the inventory viewing logic goes here.

NPC.java:

This is backend logic for NPC interactions this targets the text files that hold the strings of information for interactions. Uses Switch block to determine what file or function to access depending on the NPC type.

If the NPC is a shopkeeper, we decided to create a shop function, containing an arraylist of items that a user can purchase from. We decided to use a Java record, a class that only requires us to only declare parameter type and name. It serves to be a much simplistic way to contain data and removes the need for a hashMap and the possible walls it may introduce. This choice was done to create pairs of data that may contain similar value. For a shop, this is a valid reason as some items may contain similar prices, where a hashMap may only allow unique values

The shop function is quite simple. A user enters a shop, purchases items, and a transaction occurs. This is managed by a loop and exits when the user feels like they want to exit

Player.java:

This is the player object itself, it holds the objects that are tied to the database and allows the end user to print them to the GUI with the getDisplayToGUI method. The isPasswordCorrect method checks to see if the password entered matches what is in the database. The player is what is updated as a live session whenever a change is made within the database through the gameplay.

Puzzle.java:

This is one of the obstacles within the game, while we have the enemy class this also acts as something to hinder the player until they are able to solve the puzzle to move forward. For simplicity sake, we chose to go for a guessing game, much like hangman. Through a loop, the user is presented a “hidden” word to guess.

The puzzle words are gathered from a text file containing a set of word banks. This is then transferred onto an arraylist, in the case that words are added to the word bank. The word is placed into a String variable, the hidden version into a StringBuilder to allow for correct guesses to be added into the hidden word, while still keeping other letters hidden

SaveSelectController.java:

This file deals with the logging in and logging out, the password is storred within the database which is then connected to the DatabaseConnection, the password then follows this track until it is validated and the player can log in. Once the player has logged in they are able to access their inventory and all of their stats. Then if they every want to switch to another save slot they are able to do so because their session is saved and live updates to the database.

SceneManager.java:

This handles the switching between the different fxml scenes.

ShopController.java:

This file controls the backend logic for the shop and will allow a user to buy and sell options as well as view their inventory similar to how the mapController.java file does

Our resources folder:

This folder stors the css and .fxml files that are used to create the GUI.

Our Database:

We used SQLite for this project as to eliminate the need for a server, this helps to create a complete application. The 380DatabaseSQLiteFinal.db is our database, each table is fit for a save and each table has the ability to be live updated from the GUI with the player session previously mentioned.